AI SDK: streamText() API 参考指南

面向开发者的核心参数、返回值与流事件速查手册

streamText() 是 AI SDK Core 中用于从语言模型生成流式文本的核心函数。它专为交互式场景（如聊天机器人、实时应用）设计，支持工具调用（Tool Calling）与多步骤生成。本文档精简整理了其所有输入参数、返回值及流事件结构，便于开发者快速查阅。

最小调用示例

```
import { streamText } from 'ai';

const { textStream } = streamText({
  model: openai('gpt-4-turbo'),
  prompt: 'Design a futuristic vehicle.',
});

for await (const textPart of textStream) {
  process.stdout.write(textPart);
}
```

1. 调用输入参数 (Input Parameters)

以下参数涵盖了模型配置、消息结构、工具定义及流控制选项。

1.1 核心配置 (Core & Messages)

参数	类型	说明
model	LanguageModel	目标语言模型实例（如 `openai('gpt-4')`）。
prompt	string | Array	简单的文本提示输入，用于单轮生成。
system	string	系统指令，用于定义模型行为或角色。
messages	Array	

对话历史列表。支持多模态与工具消息。

- **User**: text, image, file (base64/url/buffer)
- **Assistant**: text, tool-call (id, name, input)
- **Tool**: tool-result (id, result, isError)

1.2 工具与执行 (Tools & Execution)

参数	说明
tools	工具定义集合。包含 `description`、`parameters` (Zod Schema) 及 `execute` 函数。若未提供 execute，则仅返回 tool calls 而不自动执行。
toolChoice	工具选择策略：`'auto'` (默认)，`'none'`，`'required'` 或指定工具 `{ type: 'tool', toolName: '...' }`。
activeTools	限制当前步骤可使用的工具子集（Array）。
maxRetries	最大重试次数，默认为 2。设为 0 可禁用。
stopWhen	多步执行的终止条件，默认为 `stepCountIs(1)`。
prepareStep	多步执行前的钩子函数。可针对每一步动态修改 `model`，`params`，`messages`。

1.3 采样与生成 (Sampling & Generation)

参数	参数
temperature	maxOutputTokens
topP	presencePenalty
topK	frequencyPenalty
seed	stopSequences

1.4 事件回调与网络 (Callbacks & Network)

参数	说明
onChunk	流式片段回调。包含 `text-delta`，`tool-call`，`reasoning` 等事件。
onFinish	整个生成过程（含所有步骤）完成时的回调。提供完整 usage 和 result。
onStepFinish	多步生成中，每完成一步（LLM 往返）时的回调。
onError	发生错误时的回调。
timeout	超时设置（毫秒）。支持数字或对象：`{ totalMs, stepMs, chunkMs }`。
abortSignal	用于手动中断请求的 `AbortSignal`。
output	结构化输出规范。 • **Output.text()**: 纯文本（默认）。 • **Output.object({ schema })**: 强制返回符合 Zod Schema 的对象。 • **Output.array({ element })**: 返回元素数组。

其他高级/实验性参数： `headers` (HTTP 头)，`providerOptions` (厂商特定参数)，`experimental_telemetry` (OpenTelemetry 设置)，`experimental_transform` (流变换)，`experimental_repairToolCall` (自动修复工具调用格式)，`experimental_download` (自定义 URL 下载逻辑)。

2. 返回值与流事件 (Return Values & Events)

函数返回一个包含 Promise 属性、数据流及辅助方法的结果对象。所有 Promise 属性在访问时会自动消费流。

2.1 结果属性 (Promise Based)

属性	类型	描述
text	Promise<string>	完整的生成文本。
content	Promise<any>	生成内容。若使用了 `Output.object`，则为解析后的对象。
usage / totalUsage	Promise<Usage>	Token 统计。含 `inputTokens`，

		`outputTokens`, `totalTokens`。
toolCalls	Promise<Array>	所有被执行的工具调用列表。
toolResults	Promise<Array>	工具执行后的返回值列表。
finishReason	Promise<string>	终止原因：'stop', 'length', 'tool-calls', 'content-filter', 'error'。
steps	Promise<Array>	多步执行的详细记录，包含每一步的 text, toolCalls 等。

2.2 数据流 (Streams)

流对象	包含事件类型 (Parts)
textStream	仅包含文本增量 (`string`)。最简流。
fullStream	包含所有详细事件对象： • **text-delta**: 文本片段 • **reasoning**: 推理过程片段 • **tool-call** / **tool-result**: 工具交互 • **source** / **file**: 引用源或生成文件 • **error** / **finish**: 状态事件

2.3 UI 与 响应辅助 (Helpers)

- **toTextStreamResponse(init?)**: 创建简单的文本流 HTTP Response。
- **toUIMessageStreamResponse(options?)**: 创建专用于 AI SDK UI 的复杂流响应（含工具/推理数据）。
- **pipeTextStreamToResponse(res)**: 将文本流管道输出到 Node.js ServerResponse。
- **pipeUIMessageStreamToResponse(res)**: 将 UI 消息流管道输出到 Node.js ServerResponse。

实践提示与建议 (Tips)

- **互斥参数**: `temperature` 与 `topP` 建议只设置其中一个，不要同时使用。
- **超时控制**: 使用 `timeout: { stepMs: ... }` 避免单步死锁，使用 `chunkMs` 检测流停滞。
- **工具选择**: `toolChoice: 'required'` 强制模型必须调用工具，适合“意图识别后分发”的场景。
- **数据消费**: 访问 `await result.text` 会自动消费流。若需同时处理流和最终结果，请直接处理流，或等待流结束后再读取属性。